

Final Project Report

Emily Dunham & Cooper Maughan

Introduction

Wildfires represent a growing threat to ecosystems, communities, and infrastructure across the United States. Understanding what drives and causes the total area burned in a given month, and whether early-season fire activity can serve as a leading indicator, has practical value for resource allocation and risk management.

This analysis uses the US Wildfire dataset from the USDA in conjunction with the US Forest Service, which contains records of wildfires reported across the US from 1992 to 2020. Each record includes the fire discovery date, containment date, fire size (in acres) geographic location, and cause classification. We aggregate these records to monthly summaries at the national level, producing two functional observations per year: a monthly fire count curve and a monthly mean area burned curve.

We ask two scientific questions:

1. Does the monthly profile of fire counts across a year predict the monthly profile of mean area burned that same year?
2. Does restricting predictions to only use past fire counts improve or degrade predictive performance compared to using the full-year fire count profile?

To answer these questions we apply the **historical functional linear model** of Malfait and Ramsay (2003), which models a functional response as a smooth function of only the past history of a functional predictor. This is a natural fit for our setting because it does not make scientific sense to use fire counts from a later month to predict area burned in an earlier month. We compare this to an unrestricted function-on-function regression model as a baseline, both fitted using the `pffr` framework in the `refund` package for R.

Methods

Functional Data Setup

Let $i = 1, \dots, n$ index years and $t \in \{1, \dots, 12\}$ index months. For each year i we observe:

- $Y_i(t)$: mean area burned (acres) in month t
- $X_i(s)$: number of fires recorded in month s

We apply a $\log(x + 1)$ transformation to both $Y_i(t)$ and $X_i(s)$ to reduce skewness before modeling.

Historical Functional Linear Model

Malfait and Ramsay (2003) introduced the historical functional linear model for settings where a functional predictor $X_i(s)$ influences a functional response $Y_i(t)$, but only through its past values. The model is:

$$Y_i(t) = \mu(t) + \int_{\mathcal{S}(t)} \beta(s, t) X_i(s) ds + \varepsilon_i(t)$$

where the integration domain $\mathcal{S}(t) = \{s : s < t\}$ enforces the historical constraint, $\mu(t)$ is a smooth intercept function, $\beta(s, t)$ is a bivariate coefficient surface defined only on the region $s < t$, and $\varepsilon_i(t)$ is a mean-zero error process. The surface $\beta(s, t)$ captures the influence that fire counts at month s have on mean area burned at month t , restricted to cases where s precedes t .

We estimate this model using the `pffr` function from the `refund` package in R, which represents $\beta(s, t)$ using penalized bivariate splines and selects smoothing parameters via neighborhood cross-validation (NCV). The historical constraint is specified via `limits = 's<t'` in the `ff()` term.

Comparison Model

As a baseline we also fit the unrestricted function-on-function regression model:

$$Y_i(t) = \mu(t) + \int_{\mathcal{S}} \beta(s, t) X_i(s) ds + \varepsilon_i(t)$$

where integration is over the full domain $\mathcal{S} = \{1, \dots, 12\}$. This model is scientifically less appropriate since it allows future fire counts to inform predictions of past area burned, but serves as a useful upper bound on in-sample fit.

Prediction and Evaluation

We split the data into a training set (1992–2015, $n = 24$ years) and a test set (2016–2020, $n = 5$ years). The unrestricted model predictions are obtained via `predict.pffr`. Because `predict.pffr` does not currently support the `limits` argument, historical model test predictions are computed manually using the estimated intercept $\hat{\mu}(t)$ and coefficient surface $\hat{\beta}(s, t)$:

$$\hat{Y}_i(t) = \hat{\mu}(t) + \sum_{s < t} \hat{\beta}(s, t) X_i(s)$$

Both models are evaluated using root mean squared error (RMSE) on the held-out test years, computed on the original (non-log) scale.

Data Analysis

Regular Model

```
FullModel <- pffr(  
  Y_log ~ ff(X_log, xind = s),  
  yind = t,  
  method = "NCV",  
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))  
)
```

```
R2_reg <- summary(FullModel)  
# R-squared of .568
```

Historical Model

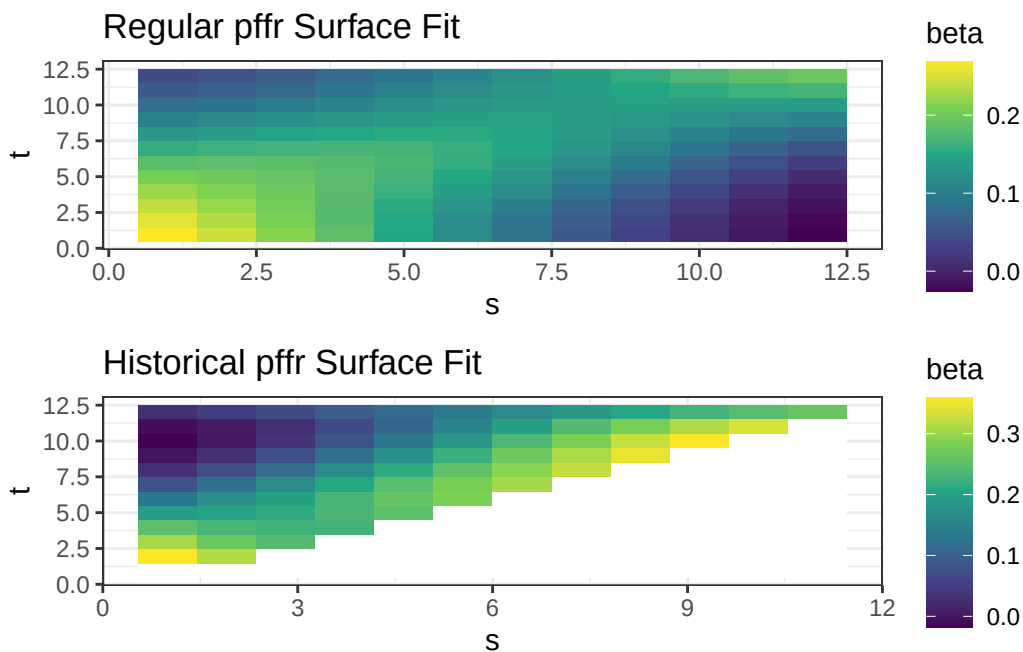
```
FullModel_Hist <- pffr(  
  Y_log ~ ff(X_log, xind = s, limits = 's<t'),  
  yind = t,  
  method = "NCV",  
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))  
)
```

```
R2_hist <- summary(FullModel_Hist)
# R-squared of .6
```

Plotting the Models

using `seWithMean` for `s(t.vec)` .

using `seWithMean` for `s(t.vec)` .



Predictions

Regular Train Model

```
Train_Model <- pffr(
  Y_log_train ~ ff(X_log_train, xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
```

```
R2_full_train <- summary(Train_Model)
# R^2 = 0.613
```

Historical Train Model

```
Train_Model_Hist <- pffr(
  Y_log_train ~ ff(X_log_train, xind = s, limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
```

```
R2_hist_train <- summary(Train_Model_Hist)
# R-squared of .637
```

Regular Model Prediction

```
X_log_test <- log(X_fires_test + 1)
reg.train_preds <- predict(
  Train_Model,
  newdata = list(X_log_train = X_log_test)
)
# Look at RMSE
y_test <- t(Y_area_test)
reg.train_preds_t <- t(reg.train_preds)
regular_rmse <- sqrt(mean((y_test - reg.train_preds_t)^2))
regular_rmse
```

```
[1] 141.591
```

Historical Model Prediction

```
# Get intercept
smterms <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[1]]$value
```

using seWithMean for s(t.vec) .

```
pterm <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$pterm[1, 1]
```

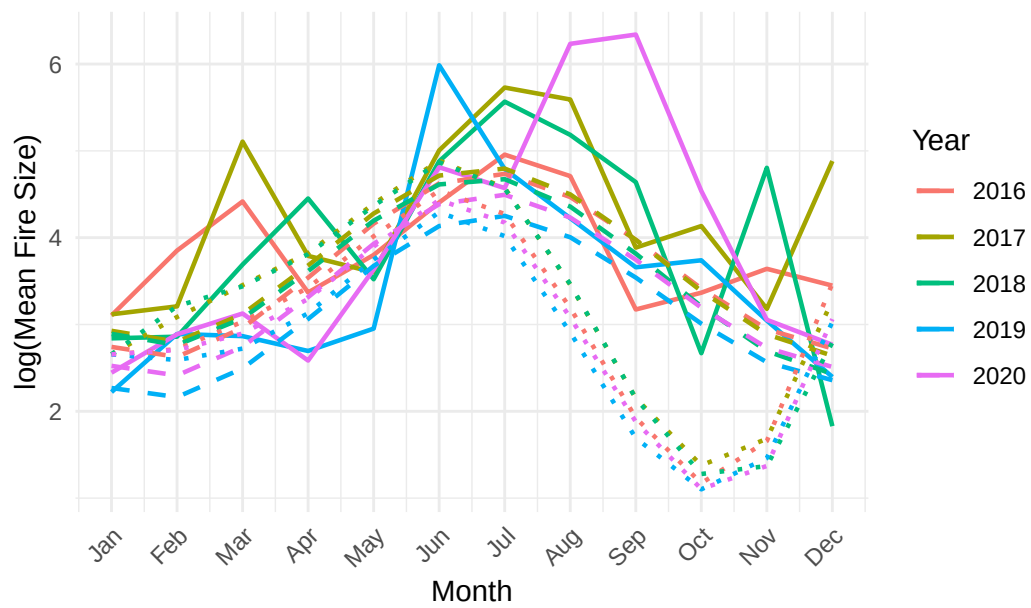
using `seWithMean` for `s(t.vec)` .

```
b0 <- pterm + smterm
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)
# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t
# Manual prediction
b1_matrix <- X_log_test %*% b1
b1_matrix_t <- t(b1_matrix)
hist.train_preds <- b0_matrix + b1_matrix_t
## Get RMSE's
historical_rmse <- sqrt(mean((y_test - hist.train_preds)^2))
historical_rmse
```

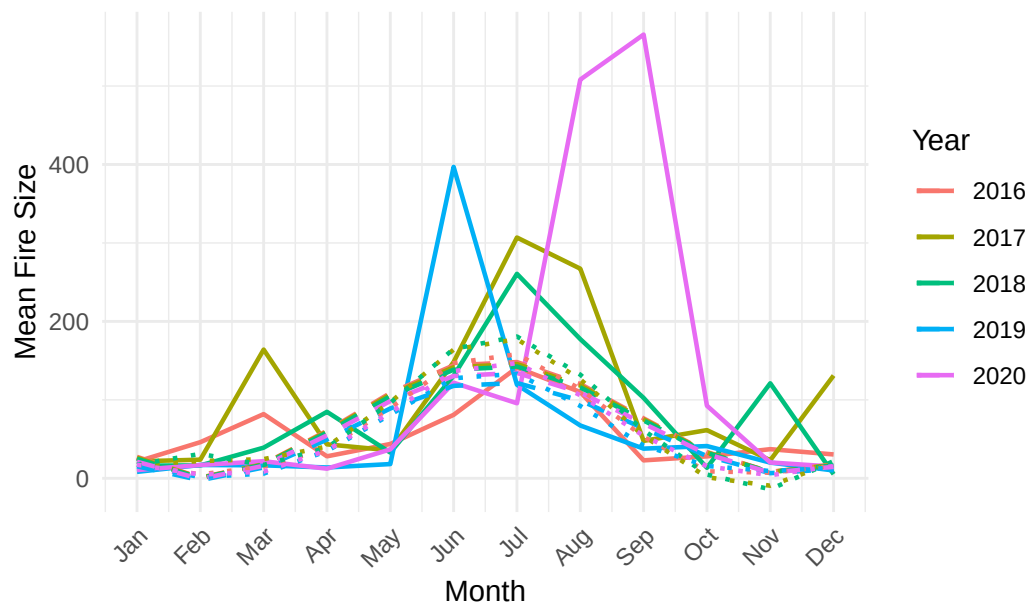
```
[1] 141.9687
```

Plot Predictions

Actual (solid) vs Regular (dashed) vs Historical (dotted) Predictions



Actual (solid) vs Regular (dashed) vs Historical (dotted)



Conclusion

What we found:

Our analysis suggests that yes, monthly fire counts do predict mean area burned, with both models explaining a meaningful portion of the variability. The regular model achieved an R^2 of 0.568 on the full data and 0.613 on the training data, while the historical model performed slightly better at 0.60 and 0.637 respectively. This is noteworthy because the historical model uses strictly less information (only past fire counts), yet predicts better. This suggests the historical constraint actually helps the model generalize rather than hurting it.

The beta surface plots tell the most interesting story. Early season fire counts (January–March) are the strongest positive predictors of area burned, showing up as bright yellow in the bottom-left corner of both surfaces. This effect decays as the year progresses. The historical surface also shows that the same-month and near-future relationships the regular model relies on are not as informative as the early season signal, reinforcing the early-season finding.

For predictions on the held-out 2016–2020 test years, both models achieved similar RMSEs (regular: 141.6, historical: 141.9), meaning neither substantially outperformed the other in actual forecasting despite the historical model fitting the training data better.

How certain we are:

We are moderately uncertain. We only have 29 years of data total, with just 5 test years, so the RMSE comparison is based on a very small test set and could easily flip with different train/test splits. The low effective rank warnings also suggest the fire count curves don't vary much in shape across years, which limits how much the functional predictor can really contribute beyond a scalar summary.

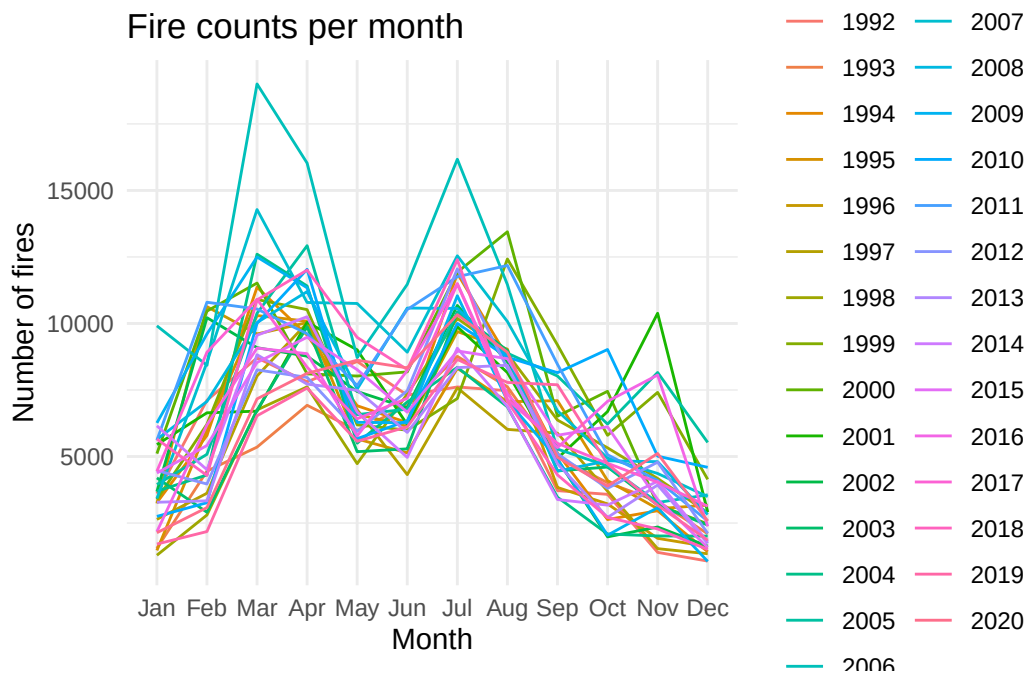
Follow-up questions worth exploring:

The most natural next question is whether adding climate variables like drought index, temperature, or precipitation as scalar predictors alongside the functional fire count predictor would improve predictions. Our plots show the model struggles most with extreme years like 2018 and 2020, which were anomalous fire seasons likely driven by climate conditions not captured in fire counts alone. We'd also ask whether predicting at a regional level rather than nationally would produce stronger relationships, since the national aggregation likely smooths out a lot of meaningful local variation.

Appendix

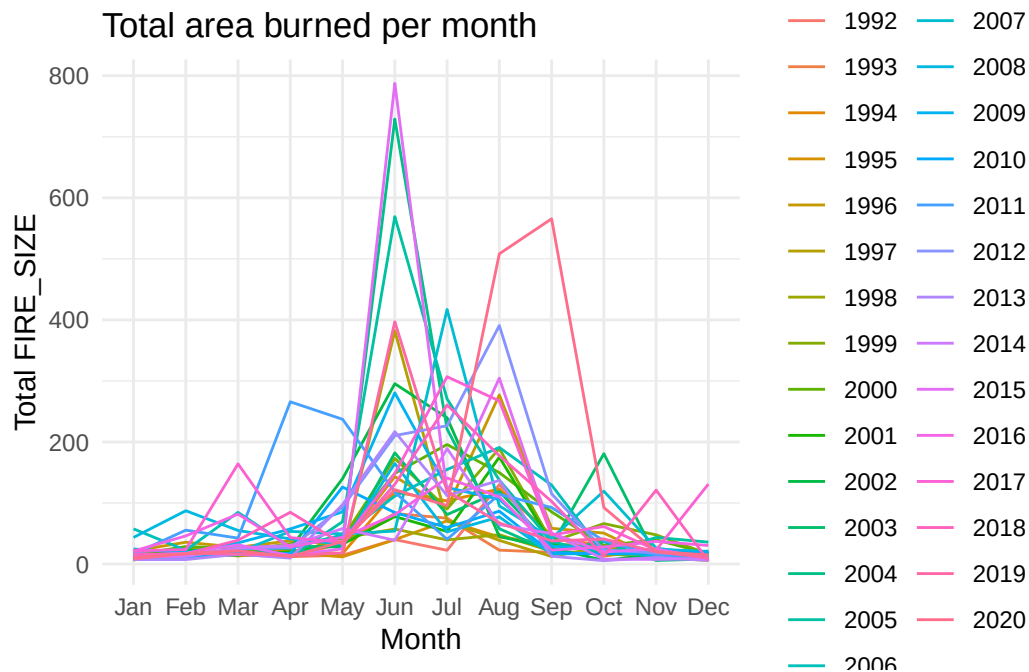
Exploratory Analysis

```
ggplot(  
  fires_counts,  
  aes(x = month, y = n_fires, color = factor(year), group = year)  
) +  
  geom_line() +  
  labs(  
    title = "Fire counts per month",  
    x = "Month",  
    y = "Number of fires",  
    color = "Year"  
  ) +  
  theme_minimal()
```



```
ggplot(  
  fires_month,  
  aes(x = month, y = total_size, color = factor(year), group = year)  
) +
```

```
geom_line() +
labs(
  title = "Total area burned per month",
  x = "Month",
  y = "Total FIRE_SIZE",
  color = "Year"
) +
theme_minimal()
```



All Code

```
library(ggplot2)
library(tidyverse)
library(refund)
library(gridExtra)
library(patchwork)

fires <- vroom::vroom("forestfires.csv")

fires_counts <- fires |>
```

```

mutate(
  year = FIRE_YEAR,
  month = month(DISCOVERY_DATE, label = TRUE)
) |>
count(year, month, name = "n_fires")

fires_month <- fires |>
mutate(
  year = FIRE_YEAR,
  month = month(DISCOVERY_DATE, label = TRUE)
) |>
group_by(year, month) |>
summarise(total_size = mean(FIRE_SIZE, na.rm = TRUE), .groups = "drop")

ggplot(
  fires_counts,
  aes(x = month, y = n_fires, color = factor(year), group = year)
) +
  geom_line() +
  labs(
    title = "Fire counts per month",
    x = "Month",
    y = "Number of fires",
    color = "Year"
  ) +
  theme_minimal()

```

```

ggplot(
  fires_month,
  aes(x = month, y = total_size, color = factor(year), group = year)
) +
  geom_line() +
  labs(
    title = "Total area burned per month",
    x = "Month",
    y = "Total FIRE_SIZE",
    color = "Year"
  ) +
  theme_minimal()

```

```

# Making the dataset for our Y matrix
# Taking the values from month column and makes them each their own column
Y_dat <- fires_month |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )
# Then I want to make this a matrix without the year variable
Y_mat <- as.matrix(Y_dat)[, -1]
# Then do the same for the X matrix
# Create the dataset
X_dat <- fires_counts |>
  pivot_wider(
    names_from = month,
    values_from = n_fires
  )
# Then I want to make this a matrix
X_mat <- as.matrix(X_dat)[, -1]
t <- 1:ncol(Y_mat) # response index (months)
s <- 1:ncol(X_mat) # predictor index (months)
# For total area burned take the log
Y_log <- log(Y_mat + 1)
# For fire counts take the log
X_log <- log(X_mat + 1)

FullModel <- pffr(
  Y_log ~ ff(X_log, xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
FullBetas <- coef(FullModel, n1 = 12, n2 = 12)$smterms[[2]]$value
R2_reg <- summary(FullModel)
# R-squared of .568

FullModel_Hist <- pffr(
  Y_log ~ ff(X_log, xind = s, limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
FullHistBetas <- coef(FullModel_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value

```

```

R2_hist <- summary(FullModel_Hist)
# R-squared of .6

FullModel2 <- coef(FullModel, n1 = 12, n2 = 12)$smterms[[2]]$coef
FullModel2_Hist <- coef(FullModel_Hist, n1 = 12, n2 = 12)$smterms[[2]]$coef
# Creating the dataframe
FullBetasdf <- data.frame(
  't' = c(FullModel2$X_log.tmat, FullModel2_Hist$X_log.tmat),
  's' = c(FullModel2$X_log.smat, FullModel2_Hist$X_log.smat),
  'beta' = c(FullModel2$value, FullModel2_Hist$value),
  'Fit' = factor(rep(
    c('Full pffr', 'Historical pffr'),
    times = c(nrow(FullModel2), nrow(FullModel2_Hist))
  ))
)
# Separating the dataframes into two
# Regular
Full <- ggplot(data = FullBetasdf[FullBetasdf$Fit == "Full pffr", ]) +
  geom_raster(aes(x = s, y = t, fill = beta)) +
  scale_fill_viridis_c() +
  xlab('s') +
  ylab('t') +
  theme_bw() +
  ggtitle('Regular pffr Surface Fit')
# Historical
HistPlotData <- FullBetasdf[FullBetasdf$Fit == "Historical pffr", ]
# Set beta to NA wherever s >= t
HistPlotData$beta[HistPlotData$s >= HistPlotData$t] <- NA
Full_Hist <- ggplot(data = HistPlotData) +
  geom_raster(aes(x = s, y = t, fill = beta)) +
  scale_fill_viridis_c(na.value = "white") +
  xlab('s') +
  ylab('t') +
  theme_bw() +
  ggtitle('Historical pffr Surface Fit')
grid.arrange(Full, Full_Hist)

```

```

area_train <- fires_month |>
  filter(year <= 2015)
area_test <- fires_month |>
  filter(year >= 2016)
fires_train <- fires_counts |>

```

```

  filter(year <= 2015)
fires_test <- fires_counts |>
  filter(year >= 2016)
## Y Matricies
# Make the training dataset a matrix for the model
Y_dat_area_train <- area_train |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )
Y_area_train <- as.matrix(Y_dat_area_train)[, -1]
# Make the test dataset a matrix
Y_dat_area_test <- area_test |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )
Y_area_test <- as.matrix(Y_dat_area_test)[, -1]
## X matricies
# Make the training dataset a matrix for the model
X_dat_fires_train <- fires_train |>
  pivot_wider(
    names_from = month,
    values_from = n_fires
  )
X_fires_train <- as.matrix(X_dat_fires_train)[, -1]
# Make the test dataset a matrix
X_dat_fires_test <- fires_test |>
  pivot_wider(
    names_from = month,
    values_from = n_fires
  )
X_fires_test <- as.matrix(X_dat_fires_test)[, -1]
t <- 1:ncol(Y_area_train) # response index (months)
s <- 1:ncol(X_fires_train) # predictor index (months)
# For total area burned take the log
Y_log_train <- log(Y_area_train + 1)
# For fire counts take the log
X_log_train <- log(X_fires_train + 1)

Train_Model <- pffr(
  Y_log_train ~ ff(X_log_train, xind = s),

```

```

yind = t,
method = "NCV",
bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
Train_FullBetas <- coef(Train_Model, n1 = 12, n2 = 12)$smterms[[2]]$value
R2_full_train <- summary(Train_Model)
# R^2 = 0.613

Train_Model_Hist <- pffr(
  Y_log_train ~ ff(X_log_train, xind = s, limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)
Train_HistBetas <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value
R2_hist_train <- summary(Train_Model_Hist)
# R-squared of .637

X_log_test <- log(X_fires_test + 1)
reg.train_preds <- predict(
  Train_Model,
  newdata = list(X_log_train = X_log_test)
)
# Look at RMSE
y_test <- t(Y_area_test)
reg.train_preds_t <- t(reg.train_preds)
regular_rmse <- sqrt(mean((y_test - reg.train_preds_t)^2))
regular_rmse

## We need to do this manually
## Predict function doesn't work with limits
# Get intercept
smterms <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[1]]$value
pterm <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$pterm[1, 1]
b0 <- pterm + smterms
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)
# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t
# Manual prediction
b1_matrix <- X_log_test %*% b1

```

```

b1_matrix_t <- t(b1_matrix)
hist.train_preds <- b0_matrix + b1_matrix_t
## Get RMSE's
historical_rmse <- sqrt(mean((y_test - hist.train_preds)^2))
historical_rmse

test_years <- 2016:2020
Y_log_test <- log(Y_area_test + 1)
# --- Actual values (long format)
actual_long <- as.data.frame(Y_log_test) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "actual") |>
  mutate(month = as.numeric(month))
# Then redo the long format with back-transformed values
reg_long <- as.data.frame(reg.train_preds) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_reg") |>
  mutate(month = as.numeric(month))
hist_long <- as.data.frame(t(hist.train_preds)) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_hist") |>
  mutate(month = as.numeric(month))
# Re-join and replot
plot_dat <- actual_long |>
  left_join(reg_long, by = c("year", "month")) |>
  left_join(hist_long, by = c("year", "month")) |>
  mutate(year = factor(year))
p1 <- ggplot(plot_dat, aes(x = month, color = year, group = year)) +
  geom_line(aes(y = actual), linewidth = 0.8) +
  geom_line(aes(y = pred_reg), linetype = "dashed", linewidth = 0.8) +
  geom_line(aes(y = pred_hist), linetype = "dotted", linewidth = 0.8) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(
    title = "Actual (solid) vs Regular (dashed) vs Historical (dotted) Predictions",
    x = "Month",
    y = "log(Mean Fire Size)",
    color = "Year"
  ) +
  theme_minimal() +

```



```

theme(axis.text.x = element_text(angle = 45, hjust = 1))

Train_Model_nl <- pffr(
  Y_area_train ~ ff(X_fires_train, xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)

Train_FullBetas_nl <- coef(Train_Model_nl, n1 = 12, n2 = 12)$smterms[[2]]$value

R2_full_train_nl <- summary(Train_Model_nl)
# R^2 = 0.292

Train_Model_Hist_nl <- pffr(
  Y_area_train ~ ff(X_fires_train, xind = s, limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2, 1))
)

Train_HistBetas_nl <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$smterms[[
  2
]]$value

R2_hist_train_nl <- summary(Train_Model_Hist_nl)
# R-squared of .321

reg.train_preds_nl <- predict(
  Train_Model_nl,
  newdata = list(X_fires_train = X_fires_test)
)

# Look at RMSE
y_test <- t(Y_area_test)
reg.train_preds_t_nl <- t(reg.train_preds_nl)

regular_rmse_nl <- sqrt(mean((y_test - reg.train_preds_t_nl)^2))
regular_rmse_nl
#104.20

# Get intercept

```

```

smterms <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$smterms[[1]]$value
pterm <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$pterm[1, 1]
b0 <- pterm + smterms
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)

# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas_nl
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t

# Manual prediction
b1_matrix <- X_fires_test %*% b1
b1_matrix_t <- t(b1_matrix)

hist.train_preds_nl <- b0_matrix + b1_matrix_t

## Get RMSE's
historical_rmse_nl <- sqrt(mean((y_test - hist.train_preds_nl)^2))
historical_rmse_nl
# 103.58

test_years <- 2016:2020

# --- Actual values (long format)
actual_long <- as.data.frame(Y_area_test) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "actual") |>
  mutate(month = as.numeric(month))

# Then redo the long format with back-transformed values
reg_long <- as.data.frame(reg.train_preds_nl) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_reg") |>
  mutate(month = as.numeric(month))

hist_long <- as.data.frame(t(hist.train_preds_nl)) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_hist") |>
  mutate(month = as.numeric(month))

```

```

# Re-join and replot
plot_dat <- actual_long |>
  left_join(reg_long, by = c("year", "month")) |>
  left_join(hist_long, by = c("year", "month")) |>
  mutate(year = factor(year))

p2 <- ggplot(plot_dat, aes(x = month, color = year, group = year)) +
  geom_line(aes(y = actual), linewidth = 0.8) +
  geom_line(aes(y = pred_reg), linetype = "dashed", linewidth = 0.8) +
  geom_line(aes(y = pred_hist), linetype = "dotted", linewidth = 0.8) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(
    title = "Actual (solid) vs Regular (dashed) vs Historical (dotted)",
    x = "Month",
    y = "Mean Fire Size",
    color = "Year"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
cowplot::plot_grid(p1, p2, ncol = 1, rel_heights = c(25, 25))

```